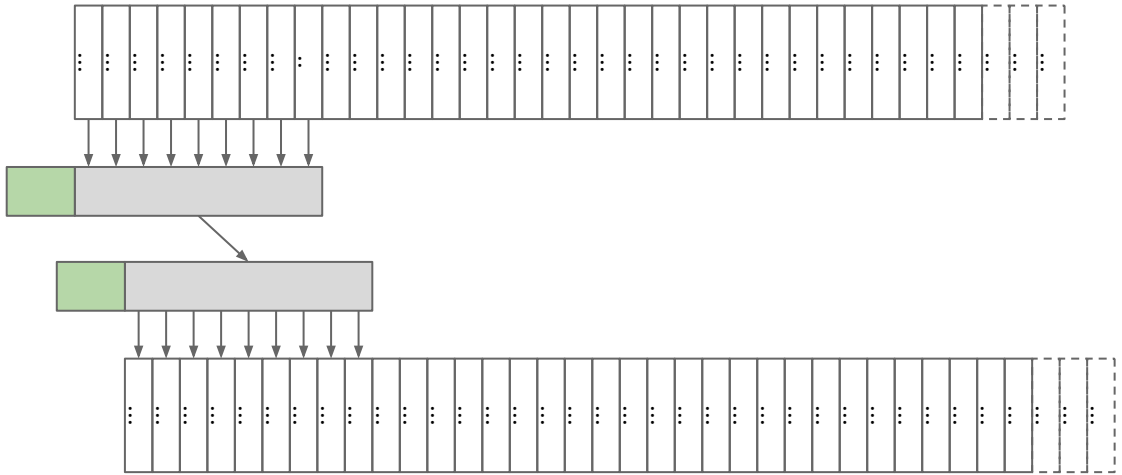


Lecture 11 (Transport 2)

TCP Implementation

CS 168, Spring 2026 @ UC Berkeley

Slides credit: Sylvia Ratnasamy, Rob Shakir, Peyrin Kao

Implementing TCP: Byte Notation (Segments, Sequence Numbers)

Lecture 11, CS 168, Spring 2026

Implementing TCP

- **Byte Notation (Segments, Sequence Numbers)**
- Maintaining State (Full Duplex, Connection Setup and Teardown)
- Sliding Window
- Header

Notation Change: Bytes vs. Packets

So far, we've used *packets* as the primary unit of data.

- Each packet has a number.
- Acks reference packet numbers.
- Window size expressed in terms of number of packets.

TCP is implemented with *bytes* as the primary unit of data.

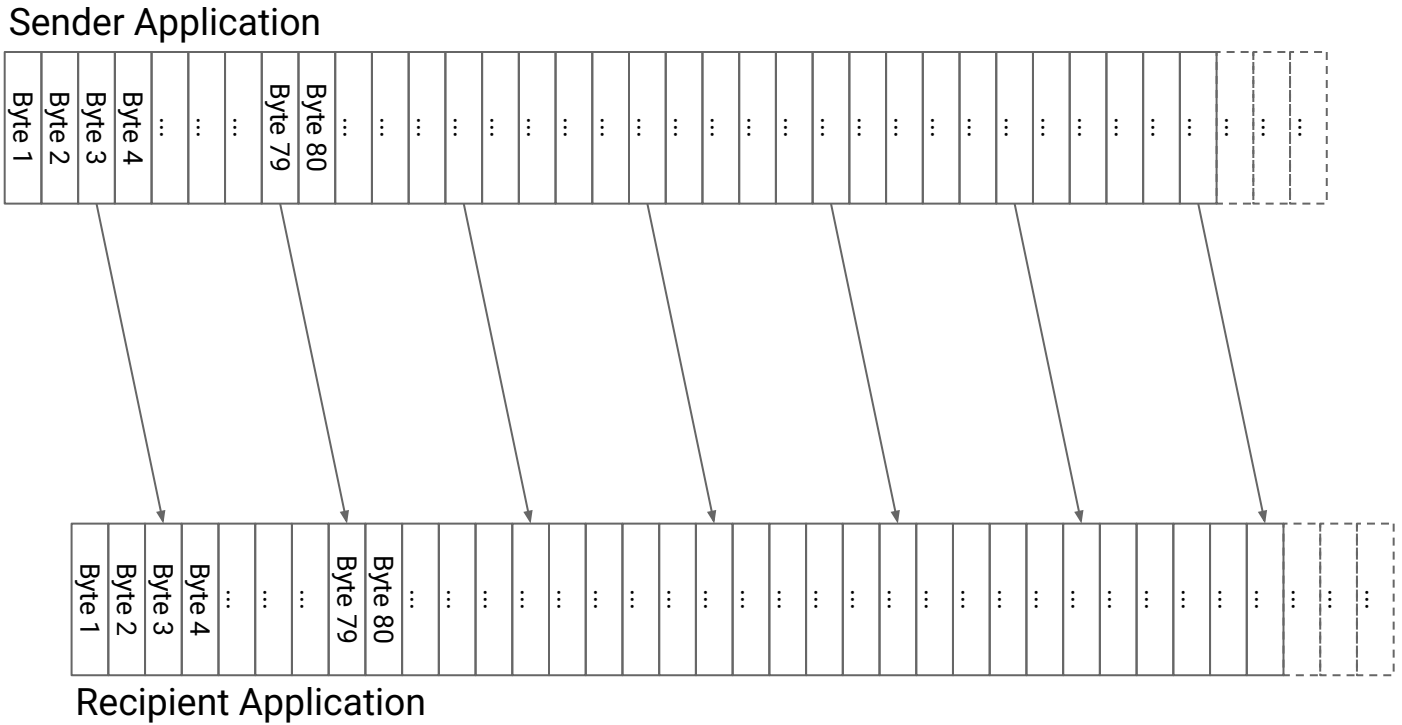
- Each byte has a number.
  Packets are defined by the number of the first byte inside.
- ACKs reference byte numbers.
- Window size expressed in terms of number of bytes.

You should be prepared to reason in terms of either.

TCP Segments

TCP provides a reliable, in-order bytestream.

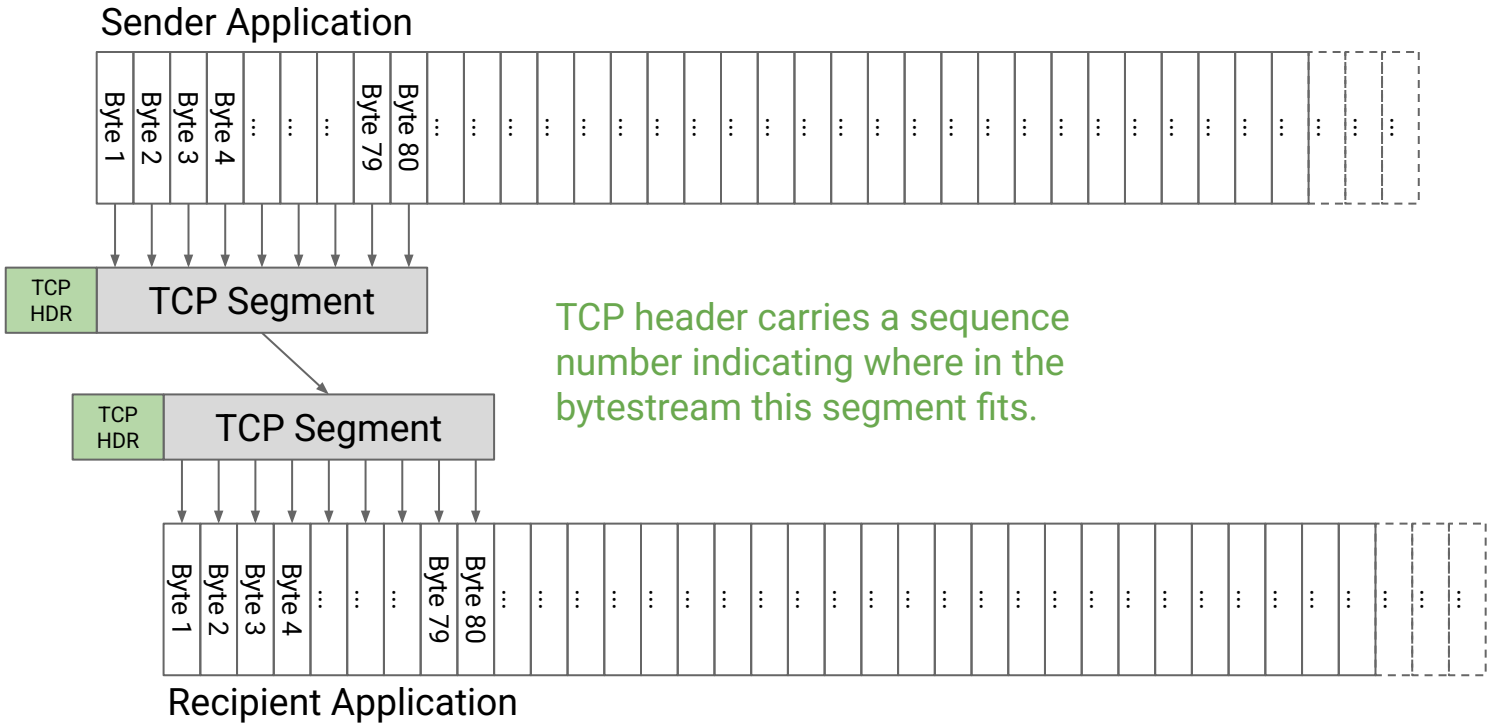
We have to split this bytestream into packets.



TCP Segments

A segment is sent when the segment is full (max segment size),

or when the segment is not full, but times out waiting for more data.

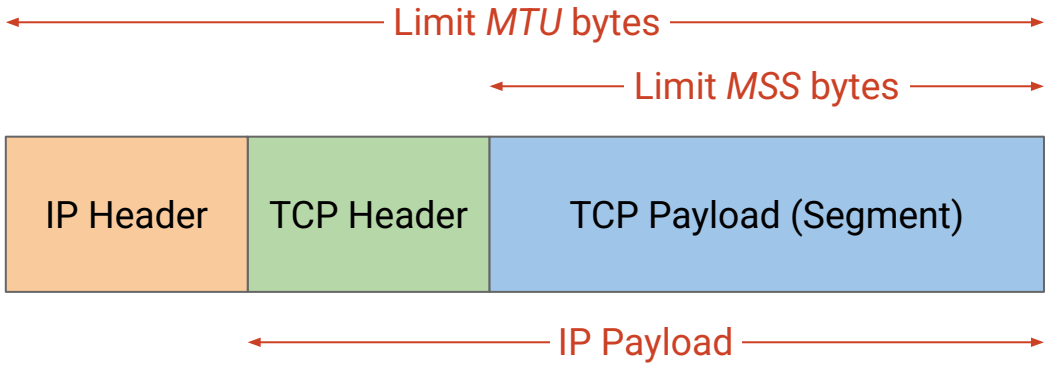


TCP Segments

TCP/IP packet: IP packet with TCP header and TCP data inside.

Size limits:

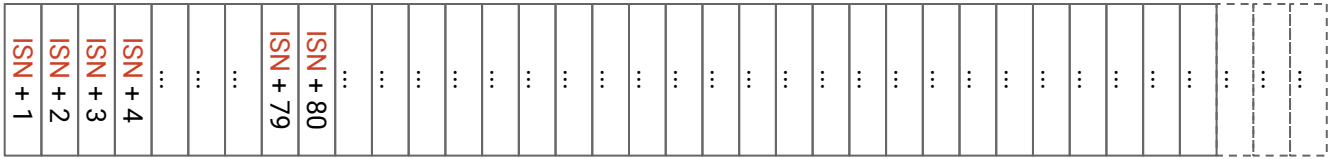
- IP packet: Maximum transmission unit (MTU).
- TCP segment: Maximum segment size (MSS).
- MSS = MTU − (IP header) − (TCP header).



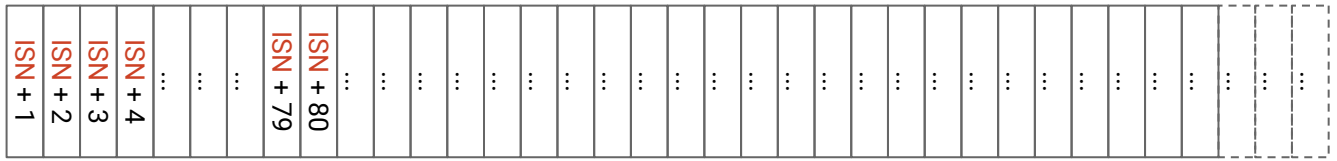
TCP Sequence Numbers

Numbering starts at a randomly-generated **Initial Sequence Number** (ISN).

- First byte is *ISN*+1, then *ISN*+2, etc.
- Starting at a randomly-chosen ISN is very important for security!



Sender Application

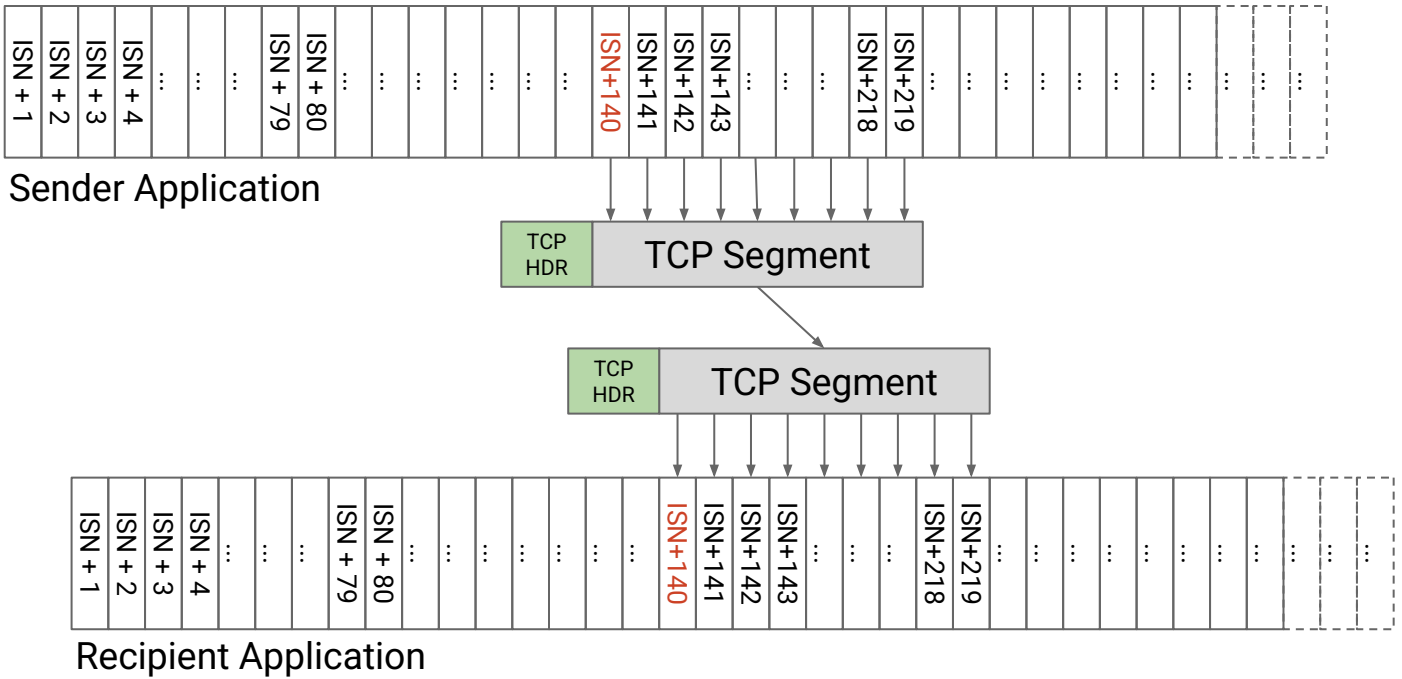


Recipient Application

TCP Sequence Numbers

The sequence number of a segment is the number of the *first byte* in the segment.

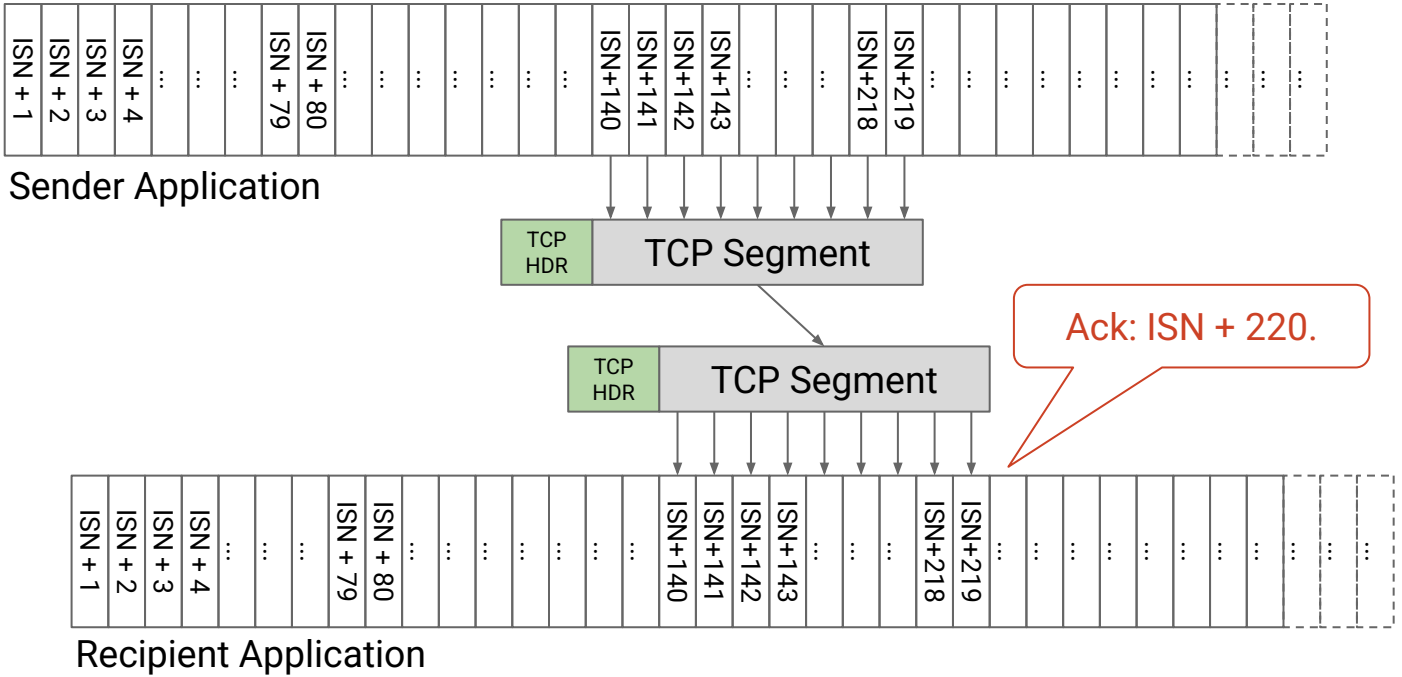
Example: In the segment below, the sequence number is *ISN* + 140.



TCP Ack Numbers

The ack number indicates the next expected byte (i.e. the first unreceived byte).

Example: All bytes up to (and including) *ISN* + 219 have been received, so the next unreceived byte is *ISN* + 220.



TCP Sequence and Ack Numbers

The sequence number of a segment is the number of the *first byte* in the segment.

- Sender sends a packet with sequence number j.
- The packet contains B bytes.
- Bytes in the packet are numbered: j, $j+1$, $j+2$, $j+3$, ..., , $j+B-1$.

Recipient sends a cumulative ack (number of highest byte received, plus one).

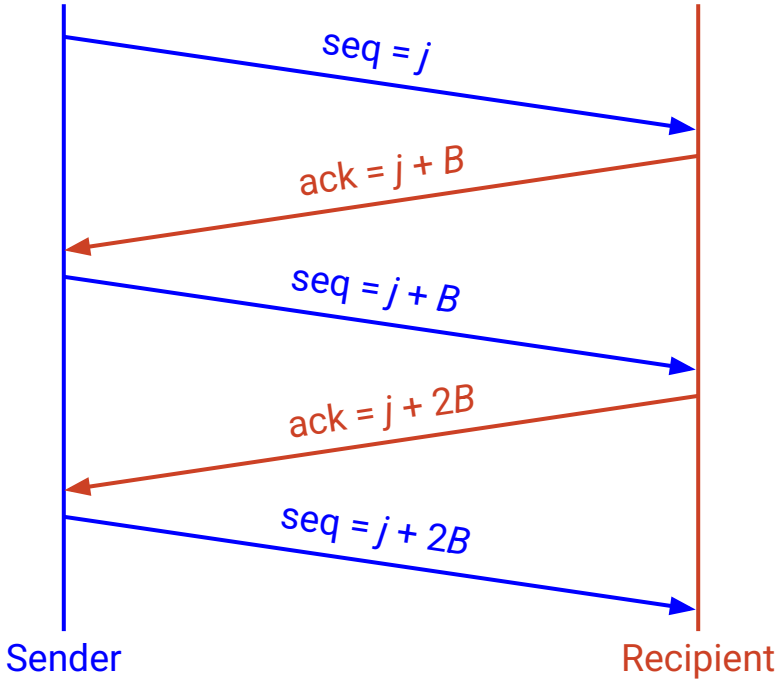
- If all prior data is received, ack number is $j+B$.
- Think of this as the next expected byte, or the first unreceived byte.
- If earlier data before this packet is missing, the ack number will be lower.

TCP Sequence and Ack Numbers

Assuming only one packet in flight, all packets length *B*, and no loss:

The last ack number is equal to the next sequence number.

- "I expect *j* + *B* next." → "I'm sending *j* + *B*."



Maintaining State (Full Duplex)

Lecture 11, CS 168, Spring 2026

Implementing TCP

- Byte Notation (Segments, Sequence Numbers)
- **Maintaining State (Full Duplex, Connection Setup and Teardown)**
- Sliding Window
- Header

TCP State

Reliability requires maintaining *state* at the end hosts.

- Sender has to remember:
  - Which packets have been sent and not acked?
  - How much longer on the timer before I resend a packet?
- Receiver has to buffer the out-of-order packets.
- State is maintained at the end hosts, not in the network.

In each separate connection, both end hosts need to maintain state.

TCP is Full-Duplex

So far, we defined a sender and a recipient in every connection.

Connections in TCP are **full-duplex**.

- Both hosts can send data, and both hosts can receive data.
- A can send to B, and B can send to A, simultaneously, in the same connection.

To support full-duplex connections:

- Two sets of sequence numbers: One for A→B bytes, and one for B→A bytes.
- Each packet carries both data and ack information.
  - "Here are some bytes starting at 15..."
  - "...Also, I ack receiving all your bytes up to (not including) 84."

15	16	17	18	19	20	21
H	e	l	l	o	,	B

A to B bytestream

77	78	79	80	81	82	83
H	e	l	l	o	,	A

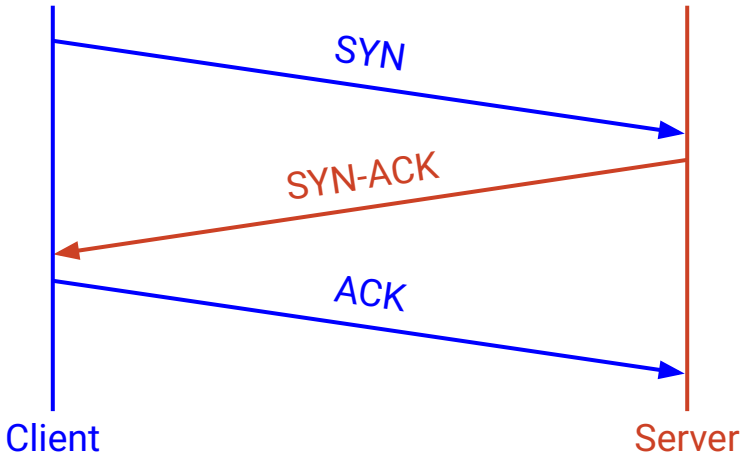
B to A bytestream

TCP Connection Setup: Three-Way Handshake

Goal: Each host tells its ISN to the other host.

1. SYN. Client says: "Here's my ISN."
2. SYN-ACK. Server says: "I received your ISN. Also, here's my ISN."
3. ACK. Client says: "I received your ISN."

After the three-way handshake, both sides can start sending data.

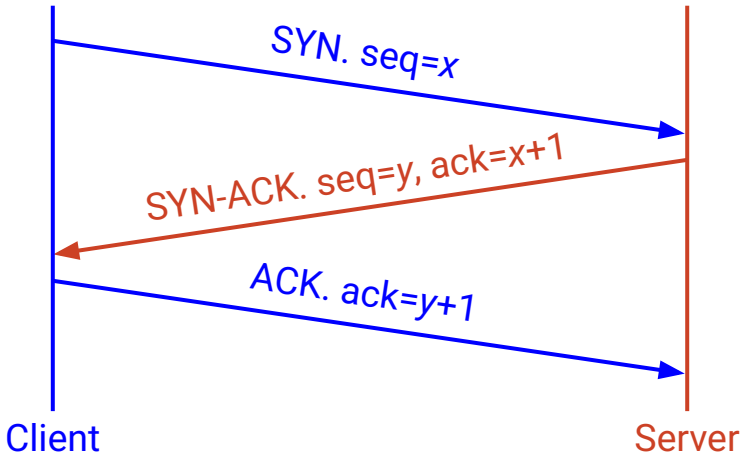


TCP Connection Setup: Three-Way Handshake

Goal: Each host tells its ISN to the other host.

1. SYN. Client says: "My ISN is *x*."
2. SYN-ACK. Server says: "I received *x* (expecting *x*+1 next). Also, my ISN is *y*."
3. ACK. Client says: "I received *y* (expecting *y*+1 next)."

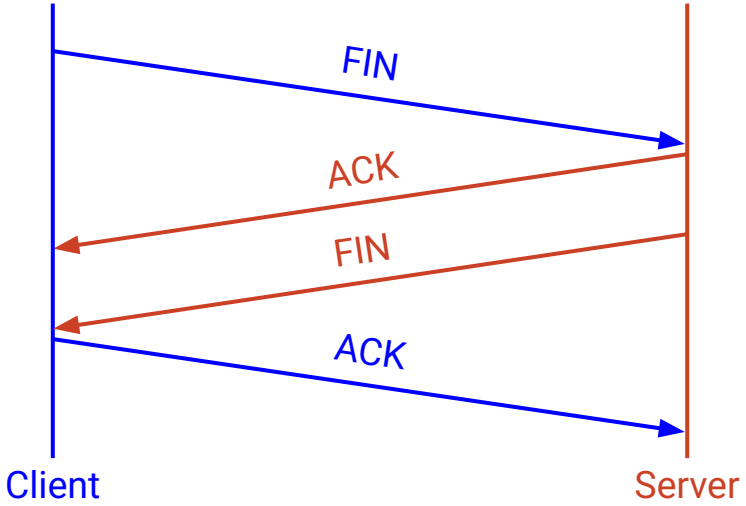
After the three-way handshake, both sides can start sending data.



TCP Connection Teardown

Normal termination:

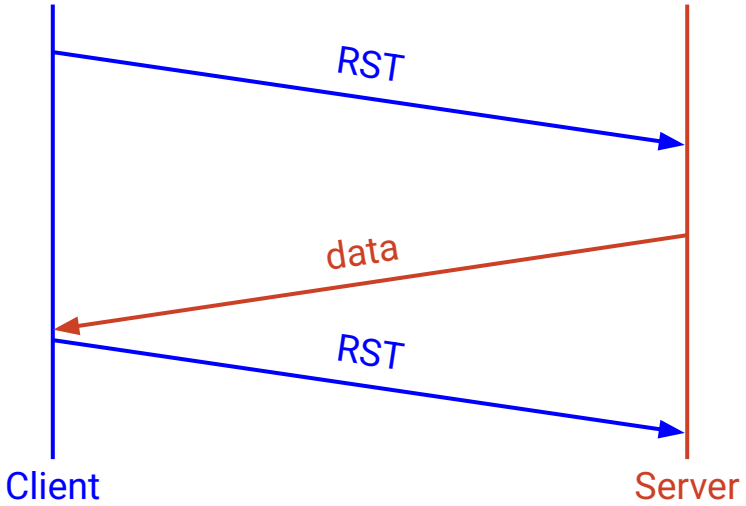
- Each side sends a FIN packet to say: "I'm done sending, but will keep receiving."
- FIN packets must be acked, just like any other data.
- When only one side has sent FIN, the connection is *half-closed*.
- When both sides have sent FIN (both done sending), the connection is closed.



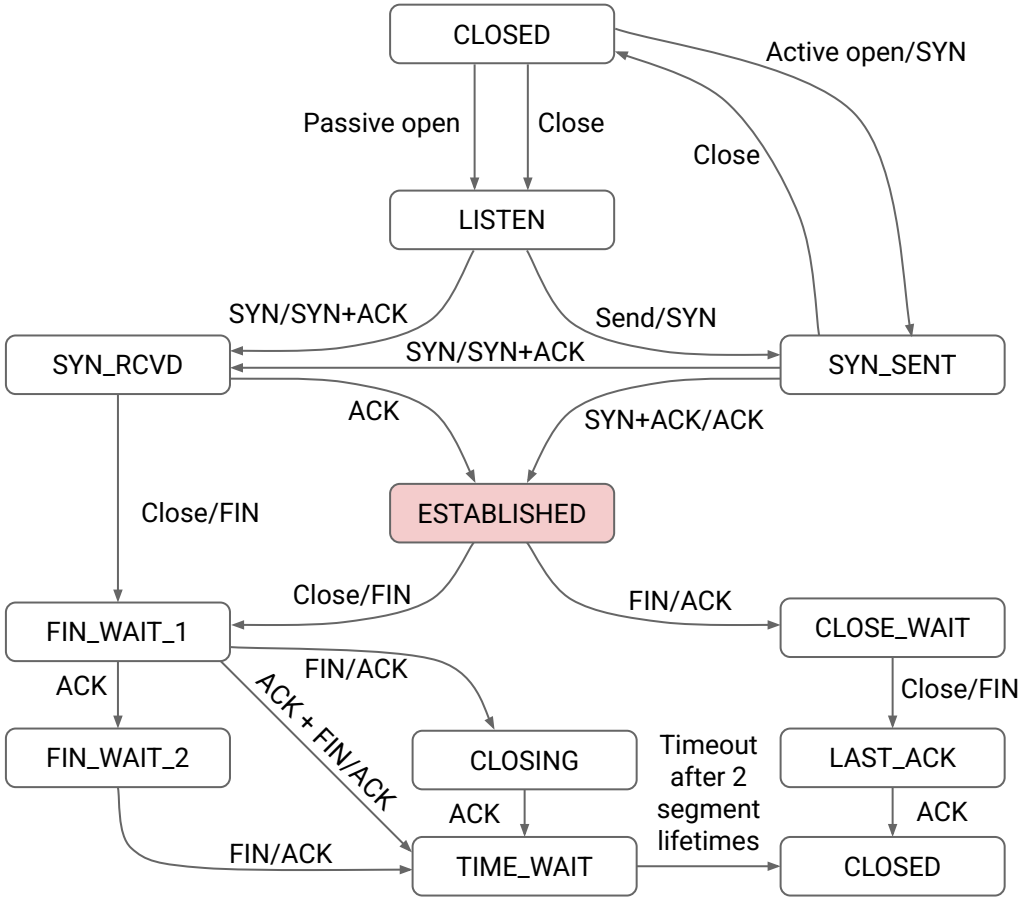
TCP Connection Teardown

Abrupt termination can be used instead (e.g. in case of error).

- Send a RST (reset) to say: "I will no longer send or receive data."
- RST packets do not need to be acked.
- Any data in flight is lost.
- If the RST sender receives more data later, send another RST.

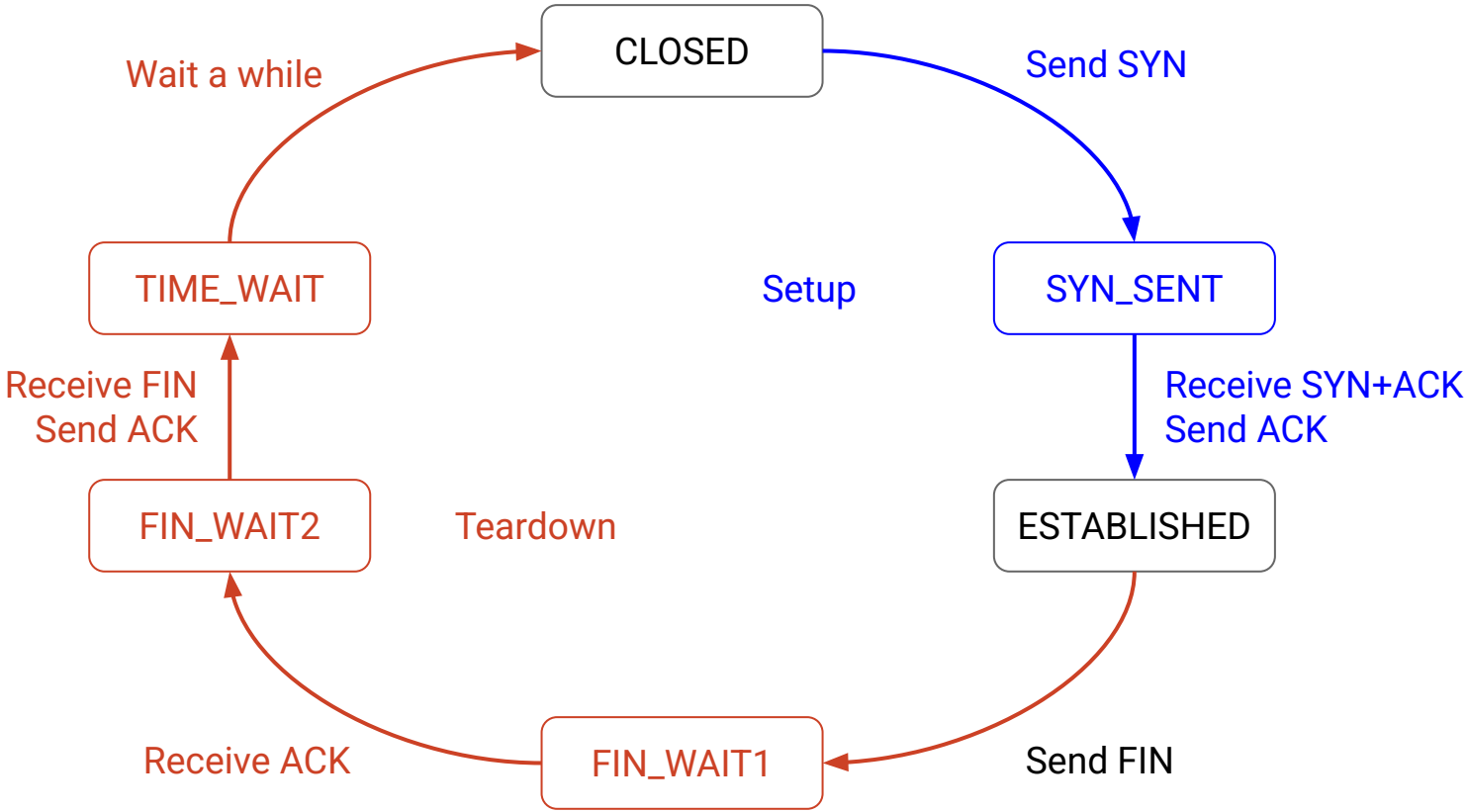


TCP Setup/Teardown Transition Diagram



The ESTABLISHED state is where all data is sent and acked.

TCP Setup/Teardown Transition Diagram (Simplified)



Piggybacking

With full-duplex, if we get a packet but have no data to send, we have two choices:

- Send the ack, with no data.
- **Piggybacking**: Wait for some data, and send the ack with the data.

Piggybacking can be tricky because TCP is in the OS, separate from the application.

- OS doesn't know when application will have more data.
- Application isn't thinking about packets and acks.

SYN-ACKs are always piggybacked.

- Ack and initial sequence number are sent together.
- Not tricky, because OS is doing the handshake, not the application.

Sliding Window

Lecture 11, CS 168, Spring 2026

Implementing TCP

- Byte Notation (Segments, Sequence Numbers)
- Maintaining State (Full Duplex, Connection Setup and Teardown)
- **Sliding Window**
- Header

TCP Sliding Window

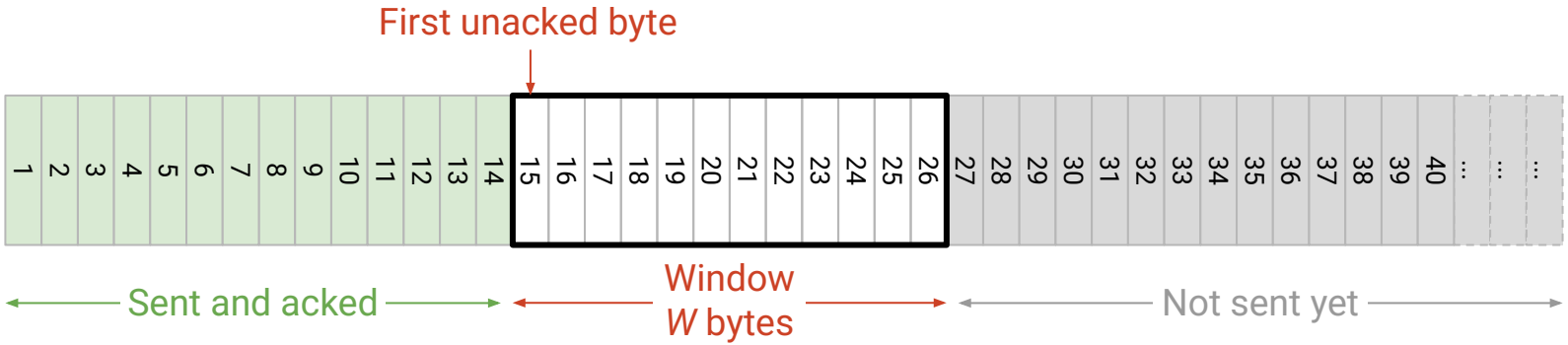
When we measured in packets, *W* was the maximum number of packets in flight.

When we measure in bytes, *W* is the maximum number of *contiguous* bytes in flight.

- The window is a range of *W* contiguous bytes, starting at the first unacked byte.
- Only these *W* bytes are allowed to be in flight.

The window slides right if and only if its *leftmost* bytes are acked.

- Example: When 15–18 arrive, we can send 27–30.



TCP Sliding Window

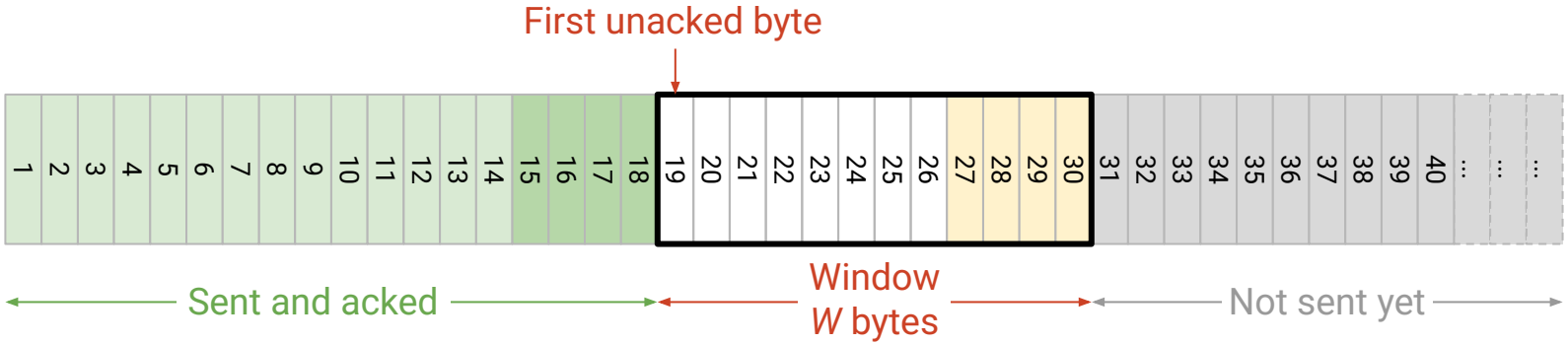
When we measured in packets, *W* was the maximum number of packets in flight.

When we measure in bytes, *W* is the maximum number of *contiguous* bytes in flight.

- The window is a range of *W* contiguous bytes, starting at the first unacked byte.
- Only these *W* bytes are allowed to be in flight.

The window slides right if and only if its *leftmost* bytes are acked.

- Example: When 15–18 arrive, we can send 27–30.



TCP Sliding Window

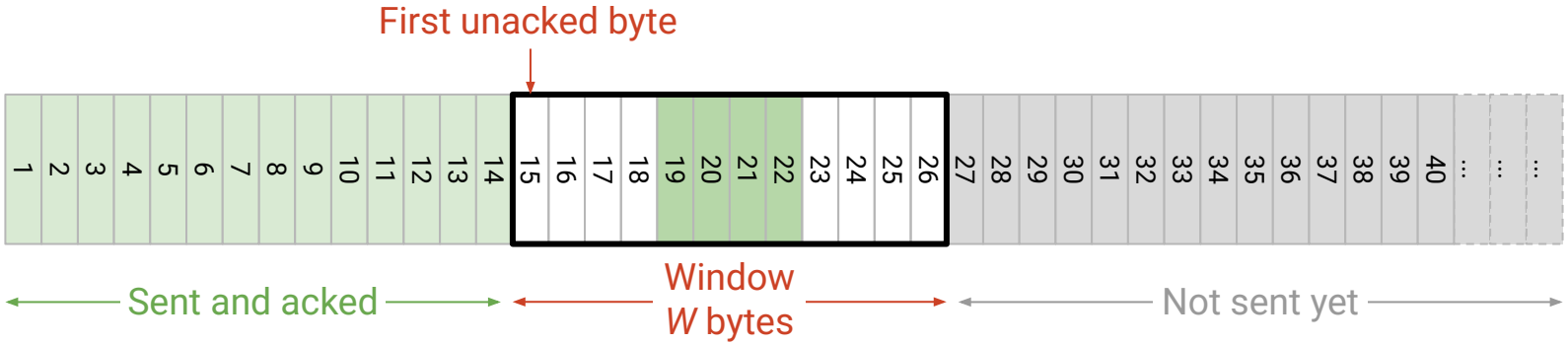
When we measured in packets, *W* was the maximum number of packets in flight.

When we measure in bytes, *W* is the maximum number of *contiguous* bytes in flight.

- The window is a range of *W* contiguous bytes, starting at the first unacked byte.
- Only these *W* bytes are allowed to be in flight.

The window slides right if and only if its *leftmost* bytes are acked.

- Acking non-leftmost bytes in the window (e.g. 19–22) does not slide the window.
- The window is determined by the first unacked byte (e.g. still 15).



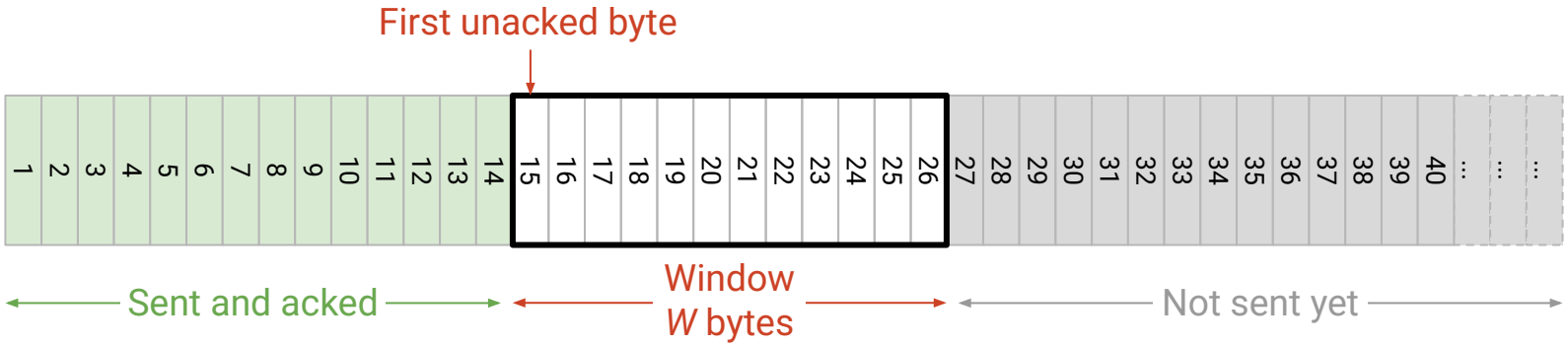
TCP Sliding Window

TCP uses cumulative acks and a sliding window.

- Cumulative ack: "I have received everything up to (not including) 15."
- Thus, first unacked byte is 15, and the sliding window is [15, 15 + *W*).

W is set as the minimum of two values:

- Advertised window (recipient reports their remaining buffer space).
- Congestion window (sender magically finds value to avoid network overload).

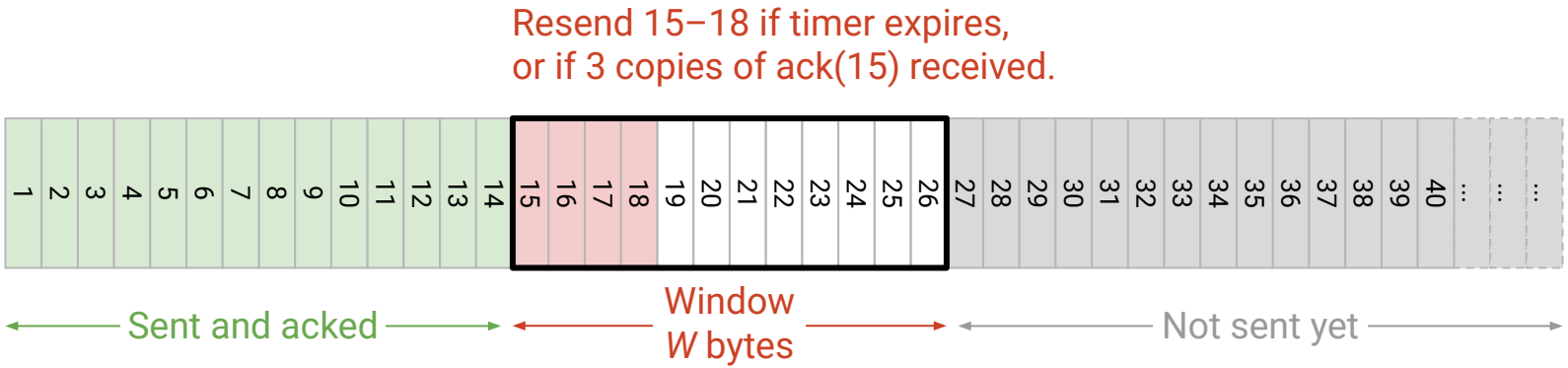


Detecting Loss

Two ways to detect loss and resend. We resend if either condition is true.

In both cases, we always resend the *first unacked packet* (leftmost part of window).

1. Ack-based: If 3 duplicate acks are received, resend.
2. Timer-based: Keep a single timer. If the timer expires, resend.
   - Different from packet-based TCP, where we kept one timer per packet.
   - Set timer by estimating RTT (e.g. measure times between packets and acks, and take a moving average).



TCP Header

Lecture 11, CS 168, Spring 2026

Implementing TCP

- Byte Notation (Segments, Sequence Numbers)
- Maintaining State (Full Duplex, Connection Setup and Teardown)
- Sliding Window
- **Header**

TCP Header

What functions does TCP implement?

1. Demultiplexing *(ports)*
2. Reliability *(checksum, sequence and ack numbers)*
3. Connection setup and teardown *(flags)*
4. Flow control *(advertised window)*

<table>
<tr><td colspan="3">Source Port (16)</td><td>Destination Port (16)</td></tr>
<tr><td colspan="4">Sequence Number (32)</td></tr>
<tr><td colspan="4">Acknowledgment Number (32)</td></tr>
<tr><td>Hdr Len (4)</td><td>0000</td><td>Flags (8)</td><td>Advertised Window (16)</td></tr>
<tr><td colspan="3">Checksum (16)</td><td>Urgent Pointer (16)</td></tr>
<tr><td colspan="4">Options (variable-length)</td></tr>
<tr><td colspan="4">Payload</td></tr>
</table>

TCP Header

There are 8 flags that can be set in TCP. We care about 4 of them:

- SYN: I'm sending my initial sequence number.
- ACK: I'm acking data (please look at the ack number).
- FIN: I'm done sending data, but will keep receiving data.
- RST: I'm done sending and receiving data.

We won't look at the other four: CWR, ECE, URG, PSH.

<table>
<tr><td colspan="3">Source Port (16)</td><td>Destination Port (16)</td></tr>
<tr><td colspan="4">Sequence Number (32)</td></tr>
<tr><td colspan="4">Acknowledgment Number (32)</td></tr>
<tr><td>Hdr Len (4)</td><td>0000</td><td>Flags (8)</td><td>Advertised Window (16)</td></tr>
<tr><td colspan="3">Checksum (16)</td><td>Urgent Pointer (16)</td></tr>
<tr><td colspan="4">Options (variable-length)</td></tr>
<tr><td colspan="4">Payload</td></tr>
</table>

TCP Header

Remaining fields:

- Header length: Measured in 4-byte words.
  - If no options, this is 5 (header is 20 bytes long).
- `0000`: Reserved bits (always set to 0).
- Urgent pointer: Used with the URG flag to indicate urgent data. Won't discuss.
- Options: Extra functionality.
  - Example: SACK uses the options field to implement full-information acks.

<table>
<tr><td colspan="3">Source Port (16)</td><td>Destination Port (16)</td></tr>
<tr><td colspan="4">Sequence Number (32)</td></tr>
<tr><td colspan="4">Acknowledgment Number (32)</td></tr>
<tr><td>Hdr Len (4)</td><td><code>0000</code></td><td>Flags (8)</td><td>Advertised Window (16)</td></tr>
<tr><td colspan="3">Checksum (16)</td><td>Urgent Pointer (16)</td></tr>
<tr><td colspan="4">Options (variable-length)</td></tr>
<tr><td colspan="4">Payload</td></tr>
</table>

TCP: Summary

An elegant (though not perfect) piece of engineering that has stood the test of time.

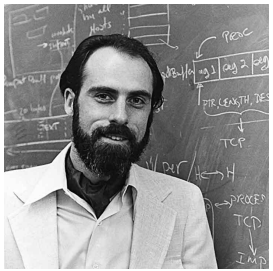
- Thought experiment: Will TCP continue to be a good solution?

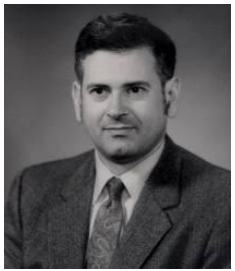
Plenty of evolution in individual pieces:

- Congestion control was added after-the-fact.
- Better acknowledgments, ISN selection, timer estimation, etc.

But the core architectural decisions and abstractions remain:

- Bytestreams, connection-oriented, windows, etc.





Vint Cerf and Bob Kahn have won basically every award ever for their work on TCP.

Lecture 12 (Transport 3)

Congestion Control, Part 1

CS 168, Spring 2026 @ UC Berkeley

Slides credit: Sylvia Ratnasamy, Rob Shakir, Peyrin Kao

Congestion Control: Why do we need it?

Lecture 12, CS 168, Spring 2026

Congestion Control Principles

- **Why do we need it?**
- Why is it hard?
- Design Goals

Dynamic Adjustment

- Algorithm Sketch
- Reacting to Congestion (AIMD)
- Implementing Congestion Control with Event-Driven Updates

Recall: Congestion at Routers

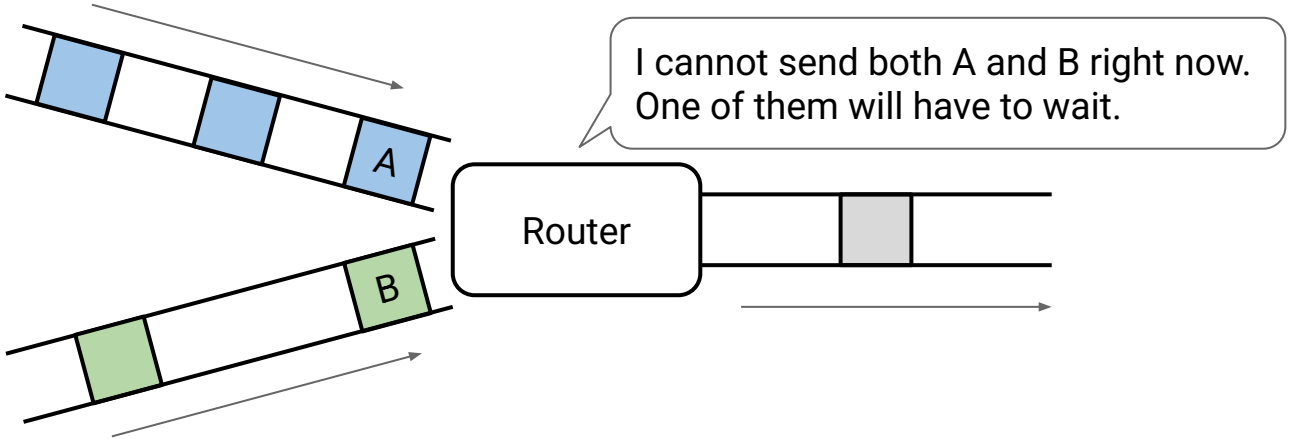
If two packets arrive at the same time, the router will send one, and buffer the other.

The router maintains a *queue* of packets that still need to be sent.

- If the queue is full, and a packet arrives, the packet gets dropped.

If too many packets arrive close in time:

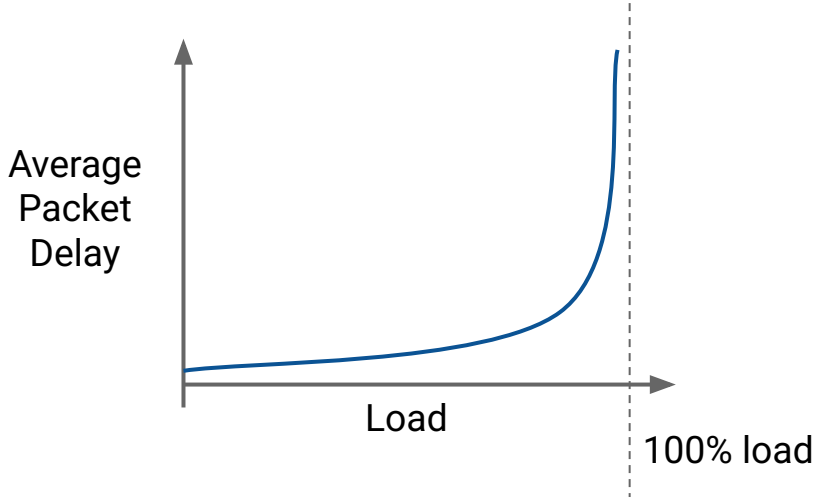
- The router cannot keep up, and gets congested.
- Packets can be delayed (stuck waiting in queue) or dropped (queue is full).



Congestion Delays Packets

For a typical queuing system with bursty arrivals:

- Higher load = more packets sent = higher packet delay.
- Delay gets really bad, even before we reach 100% load.
- Trade-offs: Need to balance high link utilization and low delay.



Brief History of Congestion Control

In the 1980s, TCP did not implement congestion control.

- Sending rate was only limited by flow control (recipient buffer capacity).
- If packets are dropped, resend them over and over, at the same fast rate.

This led to a *congestion collapse*.

- 1986: Capacity of a major link dropped to 0.1% of its original capacity.
- Why? Network was flooded with thousands of copies of every packet.

> *In October of '86, the Internet had the first of what became a series of 'congestion collapses.' During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and two IMP hops) dropped from 32 Kbps to 40 bps. We were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why things had gotten so bad. In particular, we wondered if the 4.3BSD (Berkeley Unix) TCP was mis-behaving or if it could be tuned to work better under abysmal network conditions. The answer to both of these questions was "yes".*
>
> *– Karels (UCB) and Jacobson (LBL)*

Brief History of Congestion Control

How was congestion collapse solved in the 1980s?

- Michael Karels and Van Jacobson were working on BSD (an early OS).
- They changed a few lines of TCP code to fix the problem.
  - Recall: TCP is implemented in everybody's OS.
  - The fix worked, and everybody quickly adopted it.

The history of congestion control had a big impact on its design.

- Invented as an ad-hoc patch to an existing system. (*Not in original TCP design.*)
- Implemented in the OS. (*Not in applications, routers, etc.*)

Fast-forward to today:

- Extensive research and improvements since the original patch.
- The core ideas in the original patch still persist.
- No major congestion collapses ever since.

Congestion Control: Why is it hard?

Lecture 12, CS 168, Spring 2026

Congestion Control Principles

- Why do we need it?
- **Why is it hard?**
- Design Goals

Dynamic Adjustment

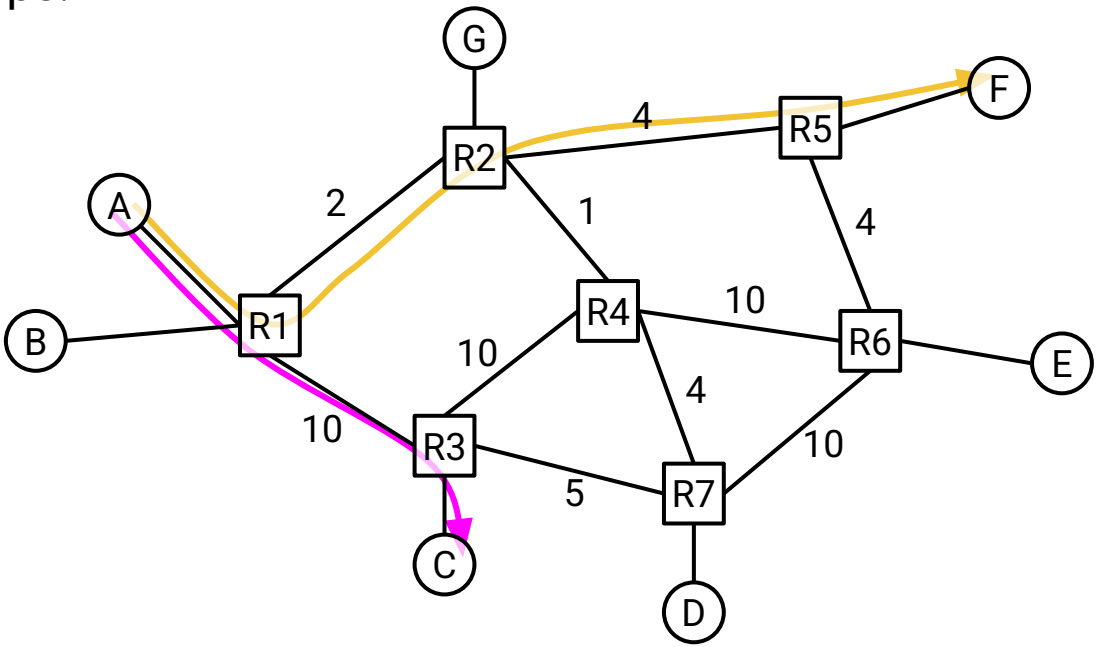
- Algorithm Sketch
- Reacting to Congestion (AIMD)
- Implementing Congestion Control with Event-Driven Updates

Congestion Control Challenges: Depends on Destination

At what rate should A send traffic?

It depends on the destination.

- A → C can send at 10 Gbps.
- A → F can only send at 2 Gbps.



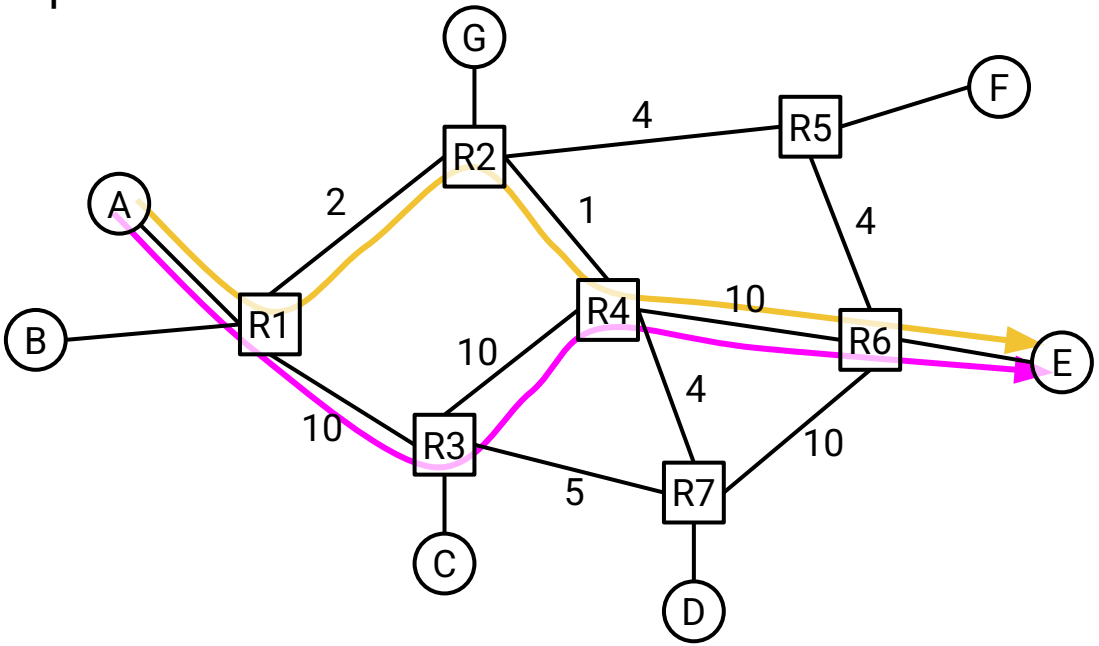
Numbers are bandwidths, in Gbps.
Ignore bandwidth of host-to-router links.

Congestion Control Challenges: Depends on Path

At what rate should A → E send traffic?

It depends on the path through the network.

- A → E via R3 can send at 10 Gbps.
- A → E via R2 can send at 1 Gbps.

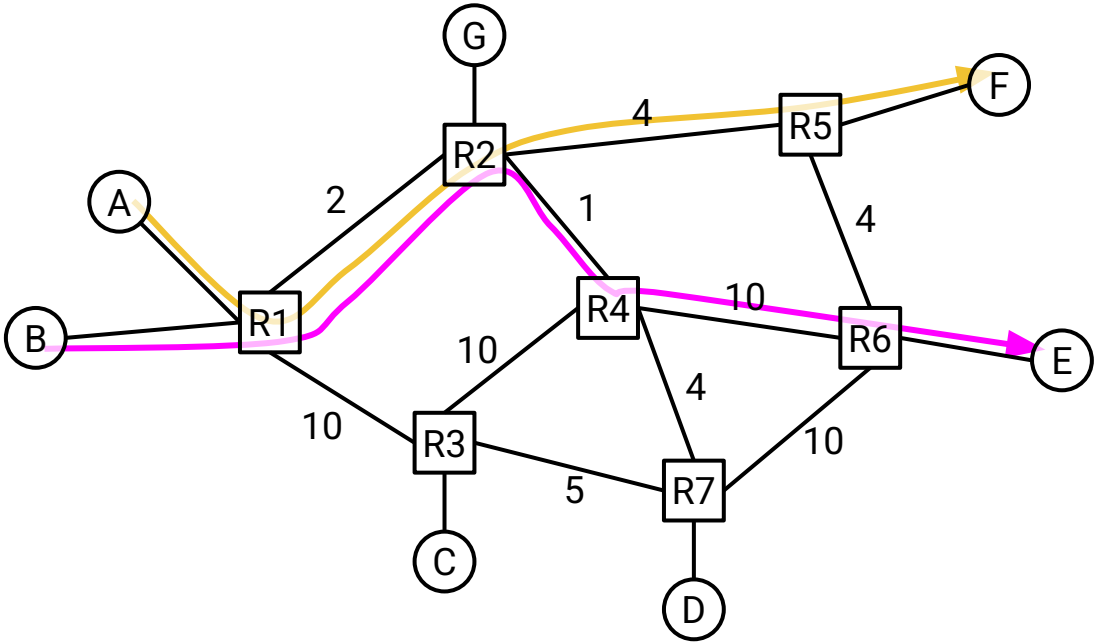


Congestion Control Challenges: Depends on Competing Flows

At what rate should A → F send traffic?

It depends on others using the network.

- A → F and B → E share a link.
- B → E can send at 1 Gbps.
- A → F can send at 1 Gbps.

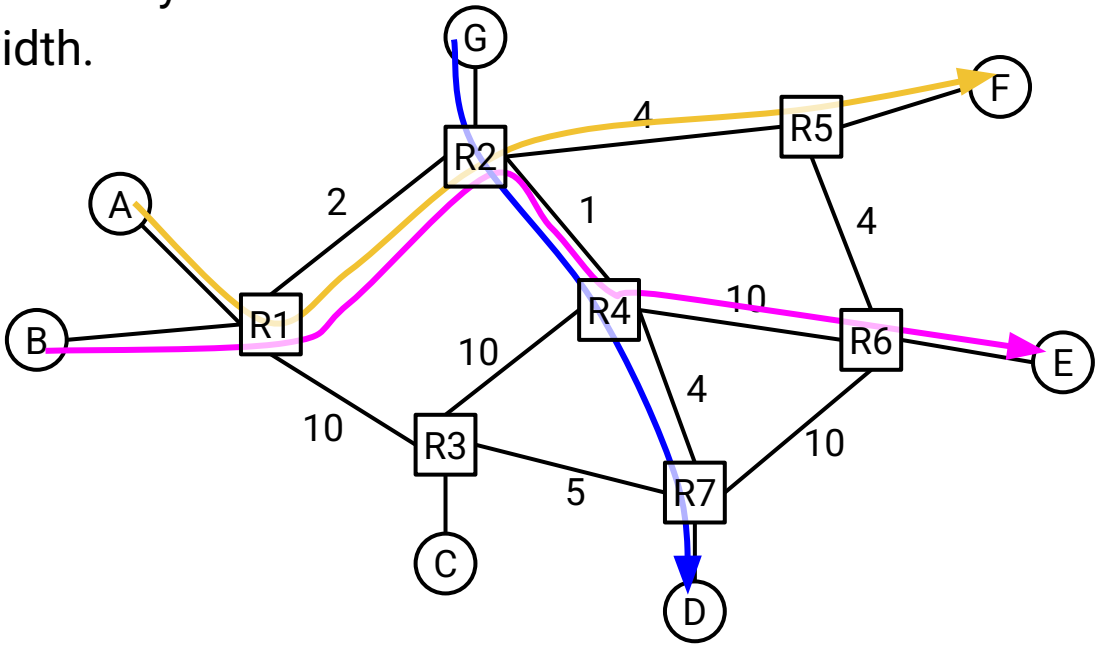


Congestion Control Challenges: Depends on Indirect Competition

What if G → D starts sending traffic?

- Maybe B → E should slow to 0.5 Gbps.
- Then A → F could increase to 1.5 Gbps.

G → D and A → F share no links, but they still affected each other's bandwidth.



Congestion Control is a Global Problem

The sender's optimal rate depends on:

- The destination.
- The path to the destination.
- Other connections sharing links along that path.
- Connections sharing links with those other connections.
- And so on...

Congestion control is a global problem.

- Connections in the network are highly interdependent.
- Solving it optimally requires a global view of the network.

Congestion Control as Resource Allocation Problem

Fundamentally, congestion control is a *resource allocation problem*.

- Bandwidth is a limited resource.
- Each connection demands a certain amount of bandwidth.
- We decide how much to allocate to each connection.

Resource allocation is a classic problem (e.g. CPU scheduling, memory allocation).

But, congestion control has additional challenges.

- A change in one connection can have a global impact.
- Network topology is constantly changing.
- Connections are constantly starting and ending.
- Everybody must solve it locally. No global view of network.

Design Goals

Lecture 12, CS 168, Spring 2026

Congestion Control Principles

- Why do we need it?
- Why is it hard?
- **Design Goals**

Dynamic Adjustment

- Algorithm Sketch
- Reacting to Congestion (AIMD)
- Implementing Congestion Control with Event-Driven Updates

Design Goals

Goals for a good congestion control algorithm:

1. Avoid congestion: Minimize packet delay and loss.
2. Efficiency: Maximize link utilization.
3. Fairness: Connections should share the available bandwidth.
4. Practical to implement: Scalable, decentralized, adaptive, etc.

Our ultimate goal is a good tradeoff between these metrics.

- Could avoid congestion (1) by sending really slowly.
- But then we aren't using all the bandwidth on the link (2).

Possible Approaches

Reservations:

- Connections reserve bandwidth at the start, and release bandwidth at the end.
- Requires connections to know bandwidth ahead of time.
- Other problems: Hard to implement, inefficient bandwidth usage, etc.

Pricing/priorities:

- Analogy: Express toll lanes on the highway. Price changes depending on traffic.
- Prioritize traffic from users who pay more.
- Requires a payment model.

Dynamic adjustment:

- Learn the current congestion level, and adjust your rate accordingly.
- Requires good citizenship (no cheating).

Possible Approaches

3 possible approaches: reservations, pricing, and dynamic adjustment.

All the algorithms we'll see are based on dynamic adjustment.

- If we rebuilt the Internet from scratch, another approach might be feasible.
- But congestion control was invented as a patch on the existing Internet.
- Dynamic adjustment is the best fit for the existing Internet:
  - Doesn't require knowing bandwidth ahead of time.
  - Doesn't require a payment model.